

MERN Stack Interview Questions & Answers

Table of Contents

1. [var, let, and const](#)
 2. [Hoisting](#)
 3. [this Keyword](#)
 4. [call, apply, and bind](#)
 5. [Event Loop](#)
 6. [Synchronous vs Asynchronous](#)
 7. [map, filter, and reduce](#)
 8. [Redux](#)
 9. [OOP in JavaScript](#)
 10. [Callbacks, Promises, and Async/Await](#)
 11. [Functions in JavaScript](#)
 12. [useEffect vs useLayoutEffect](#)
 13. [Git & GitHub](#)
 14. [Polyfills](#)
-

1. var, let, and const {#var-let-const}

What is the difference between var, let, and const?

Simple Answer:

- `var` is the old way to declare variables (before ES6)
- `let` is the modern way to declare variables that can change
- `const` is for values that should NOT change

Can you reassign or redeclare?

Feature	var	let	const
Reassign	✅ Yes	✅ Yes	❌ No
Redeclare	✅ Yes	❌ No	❌ No

Example:

```
// var - can reassign and redeclare
var name = "John";
name = "Mike";      // ✅ Works
var name = "Peter"; // ✅ Works

// let - can reassign but NOT redeclare
let age = 25;
age = 30;           // ✅ Works
let age = 35;       // ❌ Error: age already declared

// const - CANNOT reassign or redeclare
const country = "India";
country = "USA";    // ❌ Error: Assignment to constant variable
const country = "UK"; // ❌ Error: country already declared

// BUT you can modify objects/arrays with const
const person = { name: "John" };
person.name = "Mike"; // ✅ Works (changing property)
person = {};          // ❌ Error (reassigning whole object)
```

What is the scope of var, let, and const?

Scope means where you can use a variable in your code.

- **var**: Function scope (or global scope)
- **let**: Block scope { }
- **const**: Block scope { }

Example:

```

// var - Function Scope
function testVar() {
  if (true) {
    var x = 10;
  }
  console.log(x); // ✅ 10 (accessible outside if block)
}

// let - Block Scope
function testLet() {
  if (true) {
    let y = 20;
  }
  console.log(y); // ❌ Error: y is not defined
}

// const - Block Scope
function testConst() {
  if (true) {
    const z = 30;
  }
  console.log(z); // ❌ Error: z is not defined
}

// Real example
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Output: 3, 3, 3 (var is function scoped)

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 1000);
}
// Output: 0, 1, 2 (let is block scoped)

```

2. Hoisting {#hoisting}

What is hoisting?

Simple Definition: JavaScript moves variable and function declarations to the top of their scope before code runs.

Think of it like this: JavaScript reads all your declarations first, then runs the code.

How does hoisting differ for var, let, and const?

- **var:** Hoisted and initialized with `undefined`
- **let:** Hoisted but NOT initialized (Temporal Dead Zone)
- **const:** Hoisted but NOT initialized (Temporal Dead Zone)

Example:

```
// var hoisting
console.log(a); // undefined (not error!)
var a = 10;
console.log(a); // 10

// What actually happens:
var a; // Declaration hoisted
console.log(a); // undefined
a = 10; // Assignment stays here
console.log(a); // 10

// let hoisting
console.log(b); // ❌ ReferenceError: Cannot access before initialization
let b = 20;

// const hoisting
console.log(c); // ❌ ReferenceError: Cannot access before initialization
const c = 30;

// Function hoisting
sayHello(); // ✅ "Hello!" (functions are fully hoisted)

function sayHello() {
  console.log("Hello!");
}

// Function expression NOT hoisted
sayBye(); // ❌ TypeError: sayBye is not a function

var sayBye = function() {
  console.log("Bye!");
};
```

3. this Keyword {#this-keyword}

What is the value of 'this' in different contexts?

Simple Definition: `this` refers to the object that is executing the current function.

Different Contexts:

1. Global Context

```
console.log(this); // Window object (in browser)
```

2. Object Method

```
const person = {
  name: "John",
  greet: function() {
    console.log(this.name); // "John" (this = person object)
  }
};

person.greet(); // "John"
```

3. Regular Function

```
function showThis() {
  console.log(this); // Window object (in non-strict mode)
}

showThis();
```

4. Arrow Function

```
const obj = {
  name: "John",
  regularFunc: function() {
    console.log(this.name); // "John"
  },
  arrowFunc: () => {
    console.log(this.name); // undefined (this from parent scope)
  }
};

obj.regularFunc(); // "John"
obj.arrowFunc(); // undefined
```

5. Class Context

```
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    console.log(`Hello, I'm ${this.name}`);
  }
}

const john = new Person("John");
john.greet(); // "Hello, I'm John"
```

4. call, apply, and bind {#call-apply-bind}

What are call, apply, and bind?

These methods are used to **change the context (this)** of a function.

Differences:

Method	Executes Immediately?	Arguments
call	✔ Yes	Comma-separated
apply	✔ Yes	Array
bind	✘ No (returns new function)	Comma-separated

Examples:

```
const person1 = {
  name: "John",
  age: 25
};

const person2 = {
  name: "Mike",
  age: 30
};

function introduce(city, country) {
  console.log(`Hi, I'm ${this.name}, ${this.age} years old from ${city}, ${country}`);
}

// 1. call() - Execute immediately with comma-separated arguments
introduce.call(person1, "Mumbai", "India");
// Output: Hi, I'm John, 25 years old from Mumbai, India

introduce.call(person2, "Delhi", "India");
// Output: Hi, I'm Mike, 30 years old from Delhi, India

// 2. apply() - Execute immediately with array of arguments
introduce.apply(person1, ["Mumbai", "India"]);
// Output: Hi, I'm John, 25 years old from Mumbai, India

// 3. bind() - Returns a new function (doesn't execute immediately)
const johnIntroduce = introduce.bind(person1, "Mumbai", "India");
johnIntroduce(); // Call it later
// Output: Hi, I'm John, 25 years old from Mumbai, India

// Practical Example: Borrowing methods
const user1 = {
  name: "Alice",
  printName: function() {
    console.log(this.name);
  }
};

const user2 = { name: "Bob" };

user1.printName.call(user2); // "Bob" (borrowed method with different context)
```

Real-world Use Case:

```
// Finding max number in array using apply
const numbers = [5, 10, 3, 8, 1];
const max = Math.max.apply(null, numbers);
console.log(max); // 10

// Or modern way:
console.log(Math.max(...numbers)); // 10
```

5. Event Loop {#event-loop}

What is the JavaScript Event Loop?

Simple Definition: The Event Loop is like a manager that decides which code to run and when. JavaScript is single-threaded (does one thing at a time), but the Event Loop makes it feel like it can do multiple things.

How does it work?

Components:

1. **Call Stack:** Where code executes (LIFO - Last In, First Out)
2. **Task Queue (Callback Queue):** Holds async callbacks (setTimeout, events, etc.)
3. **Microtask Queue:** Holds promises (higher priority than Task Queue)
4. **Web APIs:** Handle async operations (setTimeout, fetch, etc.)

Example:

```
console.log("1. Start");

setTimeout(() => {
  console.log("2. setTimeout");
}, 0);

Promise.resolve().then(() => {
  console.log("3. Promise");
});

console.log("4. End");

// Output:
// 1. Start
// 4. End
// 3. Promise (Microtask Queue - higher priority)
// 2. setTimeout (Task Queue)
```

Visual Flow:

1. Synchronous code runs first (Call Stack)
 - "1. Start"
 - "4. End"
2. Microtask Queue runs (Promises)
 - "3. Promise"
3. Task Queue runs (setTimeout, events)
 - "2. setTimeout"

Detailed Example:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

setTimeout(() => {
  console.log("C");
}, 0);

Promise.resolve().then(() => {
  console.log("D");
}).then(() => {
  console.log("E");
});

console.log("F");

// Output Order:
// A (sync)
// F (sync)
// D (microtask - promise)
// E (microtask - promise)
// B (macrotask - setTimeout)
// C (macrotask - setTimeout)
```

6. Synchronous vs Asynchronous {#sync-vs-async}

What is the difference?

Synchronous (Sync):

- Code runs **line by line**
- Next line waits for the previous line to finish
- **Blocking** - stops other code from running

Asynchronous (Async):

- Code runs **without waiting**

- Other code can run while waiting for async operations
- **Non-blocking** - doesn't stop other code

Examples:

```
// SYNCHRONOUS EXAMPLE
console.log("Step 1");
console.log("Step 2");
console.log("Step 3");

// Output:
// Step 1
// Step 2
// Step 3
// (Runs in order)

// ASYNCHRONOUS EXAMPLE
console.log("Step 1");

setTimeout(() => {
  console.log("Step 2");
}, 2000); // Wait 2 seconds

console.log("Step 3");

// Output:
// Step 1
// Step 3
// Step 2 (After 2 seconds)
// (Step 3 doesn't wait for Step 2)
```

Real-world Example:

```
// Making coffee (Synchronous)
function makeCoffeeSync() {
  console.log("1. Boil water"); // Wait 5 min
  console.log("2. Grind beans"); // Wait 2 min
  console.log("3. Brew coffee"); // Wait 3 min
  console.log("4. Coffee ready!");
  // Total: 10 minutes
}

// Making coffee (Asynchronous)
function makeCoffeeAsync() {
  console.log("1. Start boiling water"); // Start (don't wait)

  setTimeout(() => {
    console.log("2. Water boiled!");
  }, 5000);

  console.log("3. Grinding beans"); // Do while water boils
  console.log("4. Setting up cup"); // Do other tasks
  // Total: 5 minutes (saved time!)
}
```

7. map, filter, and reduce {#map-filter-reduce}

map()

Purpose: Transform each element and return a **new array** of the same length.

Example:

```
const numbers = [1, 2, 3, 4, 5];

// Double each number
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6, 8, 10]

// Real example: Extract names
const users = [
  { name: "John", age: 25 },
  { name: "Jane", age: 30 },
  { name: "Bob", age: 35 }
];

const names = users.map(user => user.name);
console.log(names); // ["John", "Jane", "Bob"]
```

filter()

Purpose: Filter elements based on a condition and return a **new array** (smaller or same length).

Example:

```
const numbers = [1, 2, 3, 4, 5, 6];

// Get only even numbers
const evens = numbers.filter(num => num % 2 === 0);
console.log(evens); // [2, 4, 6]

// Real example: Filter adults
const users = [
  { name: "John", age: 17 },
  { name: "Jane", age: 25 },
  { name: "Bob", age: 15 }
];

const adults = users.filter(user => user.age >= 18);
console.log(adults); // [{ name: "Jane", age: 25 }]
```

reduce()

Purpose: Reduce array to a **single value** (sum, object, etc.).

Example:

```
const numbers = [1, 2, 3, 4, 5];

// Sum all numbers
const sum = numbers.reduce((total, num) => {
  return total + num;
}, 0); // 0 is the starting value
console.log(sum); // 15

// Real example: Count occurrences
const fruits = ["apple", "banana", "apple", "orange", "banana", "apple"];

const count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;
  return acc;
}, {});
console.log(count);
// { apple: 3, banana: 2, orange: 1 }
```

map vs forEach

Feature	map()	forEach()
Returns	New array	undefined
Purpose	Transform data	Just loop/iterate

Example:

```
const numbers = [1, 2, 3];

// map - returns new array
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6]

// forEach - returns nothing
const result = numbers.forEach(num => num * 2);
console.log(result); // undefined

// forEach is used for side effects
numbers.forEach(num => {
  console.log(num); // Just print, don't return
});
```

When to use which?

- **map**: When you need a **new transformed array**
- **filter**: When you need to **select certain elements**
- **reduce**: When you need **one value** from the array
- **forEach**: When you just want to **loop** and do something

Chaining Example:

```
const users = [
  { name: "John", age: 17, score: 85 },
  { name: "Jane", age: 25, score: 95 },
  { name: "Bob", age: 30, score: 75 },
  { name: "Alice", age: 22, score: 88 }
];

// Get average score of adults
const averageAdultScore = users
  .filter(user => user.age >= 18) // Get adults
  .map(user => user.score) // Get their scores
  .reduce((sum, score) => sum + score, 0) // Sum scores
  / users.filter(user => user.age >= 18).length; // Divide by count

console.log(averageAdultScore); // 86
```

8. Redux {#redux}

What is Redux?

Simple Definition: Redux is a **state management library** for JavaScript apps. It helps you manage and share data across your entire application.

Why use Redux?

- Manage global state (data needed by many components)
- Predictable state updates
- Easy debugging
- Time-travel debugging (undo/redo)

Redux Flow

Component → Action → Reducer → Store → Component

Core Principles

1. **Single Source of Truth:** One store for entire app
2. **State is Read-Only:** Can't change state directly
3. **Changes via Pure Functions:** Reducers must be pure functions

Example:

```
// 1. ACTION - What to do
const INCREMENT = 'INCREMENT';
const DECREMENT = 'DECREMENT';

// Action creators
const increment = () => ({
  type: INCREMENT
});

const decrement = () => ({
  type: DECREMENT
});

// 2. REDUCER - How to do it
const initialState = {
  count: 0
};

function counterReducer(state = initialState, action) {
  switch (action.type) {
    case INCREMENT:
      return { count: state.count + 1 };
    case DECREMENT:
      return { count: state.count - 1 };
    default:
      return state;
  }
}

// 3. STORE - Where to store data
const { createStore } = Redux;
const store = createStore(counterReducer);

// 4. DISPATCH - Send action to store
store.dispatch(increment()); // count: 1
store.dispatch(increment()); // count: 2
store.dispatch(decrement()); // count: 1

// 5. GET STATE - Read from store
console.log(store.getState()); // { count: 1 }

// 6. SUBSCRIBE - Listen to changes
store.subscribe(() => {
  console.log('State changed:', store.getState());
});
```

Real-world Example (Shopping Cart):

```
// Actions
const ADD_TO_CART = 'ADD_TO_CART';
const REMOVE_FROM_CART = 'REMOVE_FROM_CART';

const addToCart = (product) => ({
  type: ADD_TO_CART,
  payload: product
});

const removeFromCart = (productId) => ({
  type: REMOVE_FROM_CART,
  payload: productId
});

// Reducer
const initialState = {
  cart: [],
  total: 0
};

function cartReducer(state = initialState, action) {
  switch (action.type) {
    case ADD_TO_CART:
      return {
        ...state,
        cart: [...state.cart, action.payload],
        total: state.total + action.payload.price
      };

    case REMOVE_FROM_CART:
      const itemToRemove = state.cart.find(item => item.id === action.payload);
      return {
        ...state,
        cart: state.cart.filter(item => item.id !== action.payload),
        total: state.total - itemToRemove.price
      };

    default:
      return state;
  }
}

// Store
const store = createStore(cartReducer);

// Usage
store.dispatch(addToCart({ id: 1, name: "Laptop", price: 50000 }));
store.dispatch(addToCart({ id: 2, name: "Mouse", price: 500 }));
console.log(store.getState());
// { cart: [...], total: 50500 }
```

9. Object-Oriented Programming (OOP) {#oop}

What is OOP in JavaScript?

Simple Definition: OOP is a way to organize code using **objects** that have properties (data) and methods (functions).

Four Pillars of OOP:

1. **Encapsulation:** Bundle data and methods together
2. **Abstraction:** Hide complex details
3. **Inheritance:** Reuse code from parent class
4. **Polymorphism:** Same method, different behavior

Examples:

1. Classes and Objects

```
// Class - Blueprint
class Car {
  // Constructor - runs when object is created
  constructor(brand, model, year) {
    this.brand = brand;
    this.model = model;
    this.year = year;
  }

  // Method
  start() {
    console.log(`${this.brand} ${this.model} is starting...`);
  }

  getAge() {
    return 2025 - this.year;
  }
}

// Object - Instance of class
const myCar = new Car("Toyota", "Camry", 2020);
myCar.start(); // Toyota Camry is starting...
console.log(myCar.getAge()); // 5
```

2. Encapsulation

```
class BankAccount {
  #balance = 0; // Private field (# makes it private)

  constructor(accountHolder) {
    this.accountHolder = accountHolder;
  }

  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log(`Deposited: ₹${amount}`);
    }
  }

  getBalance() {
    return this.#balance;
  }
}

const account = new BankAccount("John");
account.deposit(1000);
console.log(account.getBalance()); // 1000
// console.log(account.#balance); // ❌ Error: Private field
```


3. Inheritance

```
// Parent class
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound`);
  }
}

// Child class inherits from Animal
class Dog extends Animal {
  constructor(name, breed) {
    super(name); // Call parent constructor
    this.breed = breed;
  }

  speak() {
    console.log(`${this.name} barks!`);
  }
}

class Cat extends Animal {
  speak() {
    console.log(`${this.name} meows!`);
  }
}

const dog = new Dog("Tommy", "Labrador");
const cat = new Cat("Kitty");

dog.speak(); // Tommy barks!
cat.speak(); // Kitty meows!
```

4. Polymorphism

```
class Shape {
  area() {
    return 0;
  }
}

class Circle extends Shape {
  constructor(radius) {
    super();
    this.radius = radius;
  }

  area() {
    return Math.PI * this.radius * this.radius;
  }
}

class Rectangle extends Shape {
  constructor(width, height) {
    super();
    this.width = width;
    this.height = height;
  }

  area() {
    return this.width * this.height;
  }
}

// Same method name, different implementation
const circle = new Circle(5);
const rectangle = new Rectangle(4, 6);

console.log(circle.area()); // 78.54
console.log(rectangle.area()); // 24
```

10. Callbacks, Promises, and Async/Await {#async-patterns}

1. Callback

Simple Definition: A function passed as an argument to another function, to be executed later.

Example:

```
// Simple callback
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

greet("John", function() {
  console.log("Callback executed!");
});

// Real example: Reading file
function readFile(filename, callback) {
  setTimeout(() => {
    console.log("File read: " + filename);
    callback("File content here");
  }, 1000);
}

readFile("data.txt", function(content) {
  console.log(content);
});
```

Problem: Callback Hell

```
// Nested callbacks (hard to read)
getData(function(a) {
  getMoreData(a, function(b) {
    getMoreData(b, function(c) {
      getMoreData(c, function(d) {
        console.log(d);
      });
    });
  });
});
```

2. Promise

Simple Definition: An object representing the eventual completion (or failure) of an async operation.

States:

- **Pending:** Initial state
- **Fulfilled:** Operation successful
- **Rejected:** Operation failed

Example:

```
// Creating a promise
const myPromise = new Promise((resolve, reject) => {
  const success = true;

  setTimeout(() => {
    if (success) {
      resolve("Operation successful!");
    } else {
      reject("Operation failed!");
    }
  }, 1000);
});

// Using a promise
myPromise
  .then(result => {
    console.log(result); // If resolved
  })
  .catch(error => {
    console.log(error); // If rejected
  });

// Real example: Fetching data
function fetchUser(userId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (userId) {
        resolve({ id: userId, name: "John" });
      } else {
        reject("User ID required");
      }
    }, 1000);
  });
}

fetchUser(1)
  .then(user => console.log(user))
  .catch(error => console.log(error));
```

Chaining Promises:

```
fetchUser(1)
  .then(user => {
    console.log(user);
    return fetchOrders(user.id);
  })
  .then(orders => {
    console.log(orders);
    return fetchOrderDetails(orders[0].id);
  })
  .then(details => {
    console.log(details);
  })
  .catch(error => {
    console.log(error);
  });
```

3. Async/Await

Simple Definition: Syntactic sugar over Promises that makes async code look synchronous.

Example:

```
// With Promises
function fetchData() {
  fetch('https://api.example.com/data')
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log(error));
}

// With Async/Await (cleaner!)
async function fetchData() {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log(error);
  }
}
```

Real-world Example:

```
// Sequential operations
async function makeBreakfast() {
  console.log("Starting breakfast...");

  const toast = await makeToast(); // Wait
  console.log("Toast ready:", toast);

  const coffee = await makeCoffee(); // Wait
  console.log("Coffee ready:", coffee);

  return "Breakfast ready!";
}

// Parallel operations (faster!)
async function makeBreakfastParallel() {
  console.log("Starting breakfast...");

  const [toast, coffee] = await Promise.all([
    makeToast(),
    makeCoffee()
  ]);

  console.log("Toast:", toast);
  console.log("Coffee:", coffee);

  return "Breakfast ready!";
}

function makeToast() {
  return new Promise(resolve => {
    setTimeout(() => resolve("🍞"), 2000);
  });
}

function makeCoffee() {
  return new Promise(resolve => {
    setTimeout(() => resolve("☕"), 1000);
  });
}

makeBreakfastParallel();
// Toast: 🍞
// Coffee: ☕
// (Total time: 2 seconds, not 3!)
```

Differences Summary:

Feature	Callback	Promise	Async/Await
Readability	✗ Poor (callback hell)	✓ Better	✓✓ Best
Error Handling	Each callback	.catch()	try/catch
Chaining	✗ Nested	✓ .then()	✓✓ Simple

11. Functions in JavaScript {#functions}

Types of Functions

1. Named Function

```
function greet(name) {  
  return "Hello " + name;  
}  
  
console.log(greet("John")); // Hello John
```

2. Anonymous Function

```
const greet = function(name) {  
  return "Hello " + name;  
};  
  
console.log(greet("John")); // Hello John
```

3. Arrow Function (ES6)

```
const greet = (name) => {  
  return "Hello " + name;  
};  
  
// Short form (implicit return)  
const greetShort = name => "Hello " + name;  
  
console.log(greetShort("John")); // Hello John
```

4. IIFE (Immediately Invoked Function Expression)

```
(function() {  
  console.log("I run immediately!");  
})();  
  
// With parameters  
(function(name) {  
  console.log("Hello " + name);  
})("John");
```

5. Constructor Function

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
const john = new Person("John", 25);  
console.log(john.name); // John
```

6. Generator Function

```
function* numberGenerator() {  
  yield 1;  
  yield 2;  
  yield 3;  
}  
  
const gen = numberGenerator();  
console.log(gen.next().value); // 1  
console.log(gen.next().value); // 2  
console.log(gen.next().value); // 3
```

Named Function vs Arrow Function

Feature	Named Function	Arrow Function
this binding	Has its own this	Inherits this from parent
arguments object	✓ Has	✗ No
Can be constructor	✓ Yes	✗ No
Hoisting	✓ Yes	✗ No

Example: 'this' difference


```
// Named function
const person = {
  name: "John",
  sayHi: function() {
    console.log("Hi, I'm " + this.name);
  }
};

person.sayHi(); // Hi, I'm John

// Arrow function
const person2 = {
  name: "John",
  sayHi: () => {
    console.log("Hi, I'm " + this.name); // undefined
  }
};

person2.sayHi(); // Hi, I'm undefined

// Real example: setTimeout
const user = {
  name: "John",

  regularFunc: function() {
    setTimeout(function() {
      console.log(this.name); // undefined (this = window)
    }, 1000);
  },

  arrowFunc: function() {
    setTimeout(() => {
      console.log(this.name); // John (this = user)
    }, 1000);
  }
};

user.regularFunc(); // undefined
user.arrowFunc(); // John
```

12. useEffect vs useLayoutEffect {#useeffect-vs-uselayouteffect}

useEffect

When it runs: After the browser **paints** the screen.

Use for:

- Fetching data
- Setting up subscriptions
- Manually changing DOM (non-visual)

Example:

```
import { useEffect, useState } from 'react';

function MyComponent() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("useEffect runs after paint");
    document.title = `Count: ${count}`;
  }, [count]);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

useLayoutEffect

When it runs: Before the browser **paints** the screen (synchronously).

Use for:

- Measuring DOM elements
- Synchronously re-rendering based on DOM measurements
- Preventing visual flicker

Example:

```
import { useLayoutEffect, useRef, useState } from 'react';

function MyComponent() {
  const divRef = useRef(null);
  const [height, setHeight] = useState(0);

  useLayoutEffect(() => {
    // Measure DOM before paint
    const divHeight = divRef.current.offsetHeight;
    setHeight(divHeight);
  }, []);

  return (
    <div>
      <div ref={divRef}>Some content</div>
      <p>Height: {height}px</p>
    </div>
  );
}
```

Key Differences

Feature	useEffect	useLayoutEffect
Runs	After paint	Before paint
Blocking	✗ Non-blocking	✓ Blocking
Use for	Most side effects	DOM measurements
Performance	✓ Better	Can slow down render

Visual Example (Flicker Prevention)

```
// BAD: useEffect causes flicker
function Tooltip() {
  const [position, setPosition] = useState({ x: 0, y: 0 });
  const ref = useRef(null);

  useEffect(() => {
    const rect = ref.current.getBoundingClientRect();
    setPosition({ x: rect.x, y: rect.y });
    // User sees tooltip jump from (0,0) to correct position
  }, []);

  return (
    <div ref={ref} style={{ left: position.x, top: position.y }}>
      Tooltip
    </div>
  );
}

// GOOD: useLayoutEffect prevents flicker
function Tooltip() {
  const [position, setPosition] = useState({ x: 0, y: 0 });
  const ref = useRef(null);

  useLayoutEffect(() => {
    const rect = ref.current.getBoundingClientRect();
    setPosition({ x: rect.x, y: rect.y });
    // Position set before user sees anything
  }, []);

  return (
    <div ref={ref} style={{ left: position.x, top: position.y }}>
      Tooltip
    </div>
  );
}
```

Rule of Thumb: Use `useEffect` for 99% of cases. Only use `useLayoutEffect` when you see visual flickering or need to measure DOM before paint.

13. Git & GitHub {#git-github}

What is Git?

Git is a version control system that tracks changes in your code.

GitHub is a cloud platform to store Git repositories online.

Basic Git Commands

```
# Initialize a repository
git init

# Clone a repository
git clone https://github.com/username/repo.git

# Check status
git status

# Add files to staging
git add filename.txt      # Add specific file
git add .                 # Add all files

# Commit changes
git commit -m "Added new feature"

# Push to GitHub
git push origin main

# Pull latest changes
git pull origin main

# View commit history
git log
```

Creating and Using Branches

Why branches? To work on features without affecting the main code.

```
# Create a new branch
git branch feature-login

# Switch to the branch
git checkout feature-login

# Create and switch in one command
git checkout -b feature-signup

# List all branches
git branch

# Push branch to GitHub
git push origin feature-login

# Delete a branch
git branch -d feature-login
```

Merging Branches

```
# Switch to main branch
git checkout main

# Merge feature branch into main
git merge feature-login

# If there are conflicts, resolve them manually
# Then commit the merge
git commit -m "Merged feature-login"
```

Example Workflow

```
# 1. Start on main branch
git checkout main
git pull origin main

# 2. Create feature branch
git checkout -b add-payment-feature

# 3. Work on your feature
# ... make changes ...

# 4. Add and commit
git add .
git commit -m "Added payment feature"

# 5. Push to GitHub
git push origin add-payment-feature

# 6. Create Pull Request on GitHub
# ... review and merge on GitHub ...

# 7. Update local main
git checkout main
git pull origin main

# 8. Delete feature branch
git branch -d add-payment-feature
```

Pull Request (PR)

What is it? A request to merge your branch into main branch.

Steps:

1. Push your branch to GitHub
2. Go to GitHub repository
3. Click "New Pull Request"
4. Select your branch
5. Add description
6. Request review

7. Merge after approval

14. Polyfills {#polyfills}

What is a Polyfill?

Simple Definition: A piece of code that provides modern functionality to older browsers that don't support it.

Example: `Array.prototype.map()` doesn't exist in old browsers, so we write our own version.

Polyfill for map()

```
// Check if map exists
if (!Array.prototype.map) {
  Array.prototype.map = function(callback) {
    const result = [];

    for (let i = 0; i < this.length; i++) {
      result.push(callback(this[i], i, this));
    }

    return result;
  };
}

// Usage
const numbers = [1, 2, 3];
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6]
```

Polyfill for filter()

```
if (!Array.prototype.filter) {
  Array.prototype.filter = function(callback) {
    const result = [];

    for (let i = 0; i < this.length; i++) {
      if (callback(this[i], i, this)) {
        result.push(this[i]);
      }
    }

    return result;
  };
}

// Usage
const numbers = [1, 2, 3, 4, 5];
const evens = numbers.filter(num => num % 2 === 0);
console.log(evens); // [2, 4]
```

Polyfill for reduce()

```

if (!Array.prototype.reduce) {
  Array.prototype.reduce = function(callback, initialValue) {
    let accumulator = initialValue !== undefined ? initialValue : this[0];
    let startIndex = initialValue !== undefined ? 0 : 1;

    for (let i = startIndex; i < this.length; i++) {
      accumulator = callback(accumulator, this[i], i, this);
    }

    return accumulator;
  };
}

// Usage
const numbers = [1, 2, 3, 4, 5];
const sum = numbers.reduce((total, num) => total + num, 0);
console.log(sum); // 15

```

Polyfill for bind()

```

if (!Function.prototype.bind) {
  Function.prototype.bind = function(context, ...args) {
    const fn = this;

    return function(...newArgs) {
      return fn.apply(context, [...args, ...newArgs]);
    };
  };
}

// Usage
const person = { name: "John" };

function greet(greeting, punctuation) {
  console.log(greeting + " " + this.name + punctuation);
}

const greetJohn = greet.bind(person, "Hello");
greetJohn("!"); // Hello John!

```

Polyfill for Promise.all()

```

if (!Promise.all) {
  Promise.all = function(promises) {
    return new Promise((resolve, reject) => {
      const results = [];
      let completed = 0;

      promises.forEach((promise, index) => {
        promise
          .then(result => {
            results[index] = result;
            completed++;

            if (completed === promises.length) {
              resolve(results);
            }
          })
          .catch(reject);
      });
    });
  };
}

// Usage
const promise1 = Promise.resolve(1);
const promise2 = Promise.resolve(2);
const promise3 = Promise.resolve(3);

Promise.all([promise1, promise2, promise3])
  .then(results => console.log(results)); // [1, 2, 3]

```

Polyfill for Array.includes()

```

if (!Array.prototype.includes) {
  Array.prototype.includes = function(searchElement) {
    for (let i = 0; i < this.length; i++) {
      if (this[i] === searchElement) {
        return true;
      }
    }
    return false;
  };
}

// Usage
const fruits = ["apple", "banana", "orange"];
console.log(fruits.includes("banana")); // true
console.log(fruits.includes("grape")); // false

```


Polyfill for Object.assign()

```

if (!Object.assign) {
  Object.assign = function(target, ...sources) {
    if (target == null) {
      throw new TypeError('Cannot convert undefined or null to object');
    }

    const to = Object(target);

    sources.forEach(source => {
      if (source != null) {
        for (let key in source) {
          if (source.hasOwnProperty(key)) {
            to[key] = source[key];
          }
        }
      }
    });

    return to;
  };
}

// Usage
const obj1 = { a: 1 };
const obj2 = { b: 2 };
const obj3 = { c: 3 };

const merged = Object.assign({}, obj1, obj2, obj3);
console.log(merged); // { a: 1, b: 2, c: 3 }

```

Bonus: Common Interview Questions

1. What is closure?

A closure is when a function "remembers" variables from its outer scope even after the outer function has finished.

```

function outer() {
  let count = 0;

  return function inner() {
    count++;
    console.log(count);
  };
}

const counter = outer();
counter(); // 1
counter(); // 2
counter(); // 3
// inner() still has access to count

```

2. What is the difference between == and ===?

- `==` (loose equality): Compares values after type conversion
- `===` (strict equality): Compares values AND types

```
console.log(5 == "5"); // true (converts string to number)
console.log(5 === "5"); // false (different types)

console.log(null == undefined); // true
console.log(null === undefined); // false
```

3. What is event delegation?

Attaching one event listener to a parent instead of many to children.

```
// BAD: Multiple listeners
document.querySelectorAll('button').forEach(btn => {
  btn.addEventListener('click', handleClick);
});

// GOOD: One listener on parent (Event Delegation)
document.getElementById('parent').addEventListener('click', (e) => {
  if (e.target.tagName === 'BUTTON') {
    handleClick(e);
  }
});
```

Summary Cheat Sheet

Variable Declaration

- `var`: Function scope, hoisted, can reassign/redeclare
- `let`: Block scope, not initialized when hoisted, can reassign
- `const`: Block scope, not initialized when hoisted, cannot reassign

Array Methods

- `map()`: Transform each element → new array
- `filter()`: Select elements → new array (smaller)
- `reduce()`: Combine elements → single value

Async Patterns

- **Callbacks**: Function as argument
- **Promises**: `.then()` and `.catch()`
- **Async/Await**: Looks synchronous, cleaner

React Hooks

- `useEffect`: After paint, for most side effects
- `useLayoutEffect`: Before paint, for DOM measurements

Git

- `git add .` → `git commit -m "message"` → `git push`
- Create branch: `git checkout -b branch-name`
- Merge: `git checkout main` → `git merge branch-name`

Good luck with your interview! 🚀

Remember: Practice coding examples, understand concepts deeply, and explain in simple terms.